Frontend Templates and JavaScript

Source revision: `4405d167fc323f33be76e2c7f6614fd21cc574a8`. This volume follows the browser code in dependency order: Flask injects initial values into Jinja templates; the dashboard script reads template IDs, posts a scenario, and renders response fields; the landing page has a separate preview script; the theme script updates the document and any Chart.js charts.

1. Browser architecture and contracts

The dashboard is server-rendered HTML plus client-side updates. `app/templates/index.html` receives `stations_meta`, `default_date`, `today_date`, and `default_time`. Jinja's `tojson` serializes station metadata into `window.STATIONS_META`. The form controls have stable `id` values consumed by `dashboard.js`. On `DOMContentLoaded`, the script registers listeners and immediately calls `/api/predict`. A successful response replaces displayed values and chart/table contents.

The browser does not run the ML model. Flask handles prediction and returns JSON; the browser formats and visualizes the returned values. Chart.js 4.4.1 is loaded from jsDelivr. If that CDN is unavailable, KPI rendering can still happen but chart creation raises and its local `try/catch` logs the chart failure.

Important contracts:

Template or API contract	Used by
`window.STATIONS_META[station]` has `all_lines`	`updateLineSelectOptions`
`station`, `date`, `time`, `line` JSON keys	`/api/predict` request body
`data.status`, `data.datetime`, `data.footfall`, `data.scheduling`, `data.profile_24h`, `data.travel_times`	`fetchPrediction` and renderers
HTML IDs such as `dispFootfall`, `demandChartCanvas`, `lineBreakdownTableBody`, `travelTimesTableBody`	DOM write targets
`line_curves`, `labels`, `predicted`, `historical` arrays	Chart renderers

If any template ID is renamed without updating JavaScript, the corresponding feature quietly fails or returns early. Optional chaining is used in some selectors but not everywhere; `fetchPrediction` directly reads the four controls, so missing elements cause a rejected async call.

2. `dashboard.js` module state and startup

`METRO_LINE_COLORS` is a browser-side display map. It includes an `Aqua Line` color even though this project topology/operations data may not support that line; a color entry alone does not create a route. `getLineColor` tries substring matches case-insensitively and returns a blue fallback. Because it returns the first matching entry, overlapping names can match whichever key appears earliest.

`demandChart`, `lineChart`, `currentData`, `currentTravelTimes`, and `predictionRequestId` are module-scoped mutable values shared by functions. They track Chart.js instances, latest response data, table data, and asynchronous request ordering. The `DOMContentLoaded` callback calls `initControls()` and then fetches the default selection.

`initControls()` (`dashboard.js:41-130`)

This function queries controls by ID, prevents native form navigation, and registers handlers. Station changes update the line options before fetching. Slider `input` writes the hour into the time field and visible label, then immediately fetches. Time field `change` copies its hour component into the slider, then fetches. Date change clears preset-button visual state. Line selection triggers a request. Search input only filters the already-loaded destination array; it does not call the server.

Preset button behavior: it starts from the browser's current date at noon, then repeatedly advances a day until a weekday/Saturday/Sunday condition is met. `getDay()` uses Sunday=0, unlike Python weekday numbering. Date serialization uses local date components and zero-pads month/day. `Today` does not move the date. The selected preset is marked active and a new prediction is requested. Date calculation uses browser local timezone; the app has no explicit timezone conversion here.

Potential interaction issue: every slider `input` can issue a request, not only on release. Request IDs stop stale responses from replacing newer results, but old HTTP requests are not aborted, so rapid dragging still consumes server work. The handler assumes `selectedTimeLabel` exists. `parseInt` truncates minutes from a time such as `09:45` to hour 9 only for slider sync, while the time input itself remains `09:45`.

`updateLineSelectOptions()` (132-152)

It reads current station and `window.STATIONS_META`. It resets the line dropdown to an all-lines option, then appends one `<option>` per station line. Using `document.createElement` plus `.textContent` is safer than concatenating line names into HTML. It toggles the group. It does not restore the previously selected line after rebuilding, so station change returns to station overview. If required elements/metadata are absent it returns without feedback.

3. `fetchPrediction()` async walkthrough (`154-201`)

The exact function is in Appendix A. Each call increments `predictionRequestId` and captures its own ID. It reads control values, shows a loader/status, and sends a JSON POST with `Content-Type: application/json`. `await fetch` pauses this async function while other browser work proceeds; `await res.json()` parses the body. The function does not check `res.ok`, so an HTTP error with JSON body follows the same `data.status` logic; a non-JSON error becomes a caught parse exception.

Before rendering, it compares the captured ID with the latest global ID. If another request started after it, the older response returns early. This is stale-response protection, not cancellation. A success updates `currentData`, calls `renderDashboard`, adds the ready CSS class, clears form status, and schedules chart resize on the next animation frame. A non-success shows the server's `message` or a fallback. `catch` logs details to the developer console but shows a generic network error. `finally` hides the loader only if this is still the newest request, preventing an old request from hiding the new request's spinner.

Example: request A gets id 4. User changes time before A finishes, request B gets id 5. If A resolves, `4 !== 5`, so it returns without drawing; B owns the visible state. Both requests still reached the server.

4. `renderDashboard(data)` response-to-UI mapping (`203-313`)

It first requires `data.scheduling`. It destructures common response sections and reads `scheduling.headline`. It chooses a demand value from finite `headline.line_demand` or falls back to total `footfall.hourly_demand`. If a line filter is selected, the headline can be a line-level result. Text is inserted using `textContent` for station names, explanatory strings, calendar labels, and KPI values, which avoids interpreting those values as HTML.

The function derives a human date label by constructing noon in the browser from the response date. Noon avoids many midnight/DST date shifts, but uses local browser timezone. It uses service closure state from `datetime.service_open === false`; closed scenario labels suppress regular demand, utilization, waits and trains.

KPI mapping:

• Footfall uses selected line demand if finite; instant interpolation is scaled by selected-line share. This is display arithmetic, not a separate line-specific model call.

• Headway shows `Varies` for station summary and a number for one line.

• Trains/hour likewise displays combined schedule values for summaries or one-line value.

• Utilization context reads recommended clearance capacity and labels scope as all lines or selected line.

• Wait and fleet text come from the schedule headline.

• Explanation goes to `textContent`.

Each major rendering group has its own `try/catch`, so a chart/table problem need not prevent later sections. Failures only go to console; the user may see a blank/stale section with no visible error state. `currentData` is set before renderer calls. Selected-line chart series use the corresponding line curve; the historical profile is multiplied by a share inferred from current hourly demand. This assumes the current share is suitable for every hour, though line demand shares may vary by time.

5. Chart rendering: `renderDemandChart(profile,currentHour)` (`315-405`)

It exits without canvas/profile, reads labels and numeric arrays with empty-array defaults, and creates arrays of point radius/background so the chosen hour is highlighted red and larger. It configures predicted demand and synthetic training-period baseline datasets; if multiple line curves exist, it adds dashed line-specific datasets colored through `getLineColor`.

Before replacement it destroys the previous chart instance. This avoids duplicate canvas listeners/resources. `new Chart(ctx, {type:'line', data, options})` makes a responsive line chart with custom legend, tooltip, zero-based y-axis and operating-hour x-axis. `tension: .35` visually smooths segments; it does not mean the model itself interpolated those points. A Chart.js rendering curve may visually imply intermediate values not present in the discrete data.

Risk: the selected-hour styling checks `profile.hours[i] === currentHour`; if types differ (string vs number), no point will be highlighted. CDN/version/runtime errors are handled by `renderDashboard`'s chart-level catch.

6. Breakdown and destination renderers

`renderInterchangeBreakdown(scheduling)` (`407-489`)

The section is deliberately shown for both interchange and single-line stations. It updates title/description, clears `<tbody>`, then builds rows from `scheduling.all_lines`. It makes arrays for doughnut-chart labels, demands, and colors. `status_action === 'SERVICE_CLOSED'` changes several cells to "Closed" or em dash. The `toLocaleString()` calls make demand easier to read. It then destroys/replaces the doughnut chart and gets a border color from CSS custom property `--surface` so the slices blend with the theme.

Data safety risk: the row is constructed with `tr.innerHTML` and interpolates `l.line`, `l.crowding_color`, and numeric fields from the server. Those values are currently derived from project data/model logic, but `innerHTML` is an HTML injection sink if a future untrusted value can reach them. Safer improvement: construct cells and assign `.textContent`; validate colors against an allowlist. Chart arrays use values as chart data; malformed response values may break Chart.js.

`renderTravelTimes(times)` (`491-496`)

When an array is supplied, it replaces module state, then delegates display to the filter function. When undefined, it preserves the old state and rerenders current data.

`filterAndDisplayTravelTimes()` (`498-544`)

It reads table/search/count elements, lowercases and trims the query, and filters destination and line names using substring inclusion. It supports both current API key names (`destination`, `line`, `travel_time_min`) and legacy CSV-like names (`End Station`, `Line`, `Travel Time (minutes)`). It updates filtered count, empties the table, and renders either a no-results row or destination rows.

The function also uses `innerHTML` with interpolated destination, line, and query values. This creates an injection risk if those fields become attacker-controlled. The no-results message embeds user search text. Use DOM-created elements and text nodes to render safely. The `??` operator for minutes means zero is preserved, unlike `||`. Search has O(n) time per keystroke over destination count and O(k) DOM construction for k matches; no debounce is used.

7. Landing page script: `landing.js`

An immediately invoked function expression `(function(){...})()` keeps helpers and `profile` private instead of creating globals. `drawPreviewChart` reads `window.METROINSIGHTS_PREVIEW`, normalizes a profile into SVG points using min/max, sets polyline/area geometry and marks index 4 as a 09:00 sample. The fixed index assumes labels are ordered and index 4 is the selected hour; it is sample presentation logic rather than dynamically matched time.

`initScrollMetro` computes document scroll progress between 0 and 1, moves a decorative train along a rail, and grows a progress line. It disables travel when `prefers-reduced-motion: reduce` is set. It attaches passive scroll and resize listeners. `initReveals` uses `IntersectionObserver` to add `is-visible` when content enters the viewport, or immediately reveals all items if unsupported/reduced motion. Once revealed, each item is unobserved. `initMobileNavigation` toggles `hidden`, text glyph, accessible label and `aria-expanded`; clicking a menu link closes it. DOMContentLoaded initializes all four behaviors.

8. Theme script: `theme.js`

It is another private IIFE. `applyTheme` restricts theme to `dark` or `light`, writes `documentElement.dataset.theme` (CSS uses it as a selector), and tries to persist `metroflow-theme` in localStorage. Storage failure is caught so current-page theme still works. For each theme button it updates label/title/ARIA and inserts a known static SVG icon. If Chart.js is loaded, it updates chart defaults and existing chart scale text/grid/legend colors, then calls `chart.update('none')` to redraw without animation. On DOM ready it applies the template's theme (already initialized from localStorage inline to avoid flash), then toggles when a button is clicked.

9. Templates and their roles

`app/templates/index.html` (234 lines)

Dashboard shell: head loads core CSS, shared theme CSS, then polish overrides, plus Chart.js. The order matters: later styles can override earlier selectors. It serializes `stations_meta` to JS, builds station/date/time/line controls, and supplies IDs consumed by `dashboard.js`. `defer` on theme JS delays execution until parse completion; dashboard.js is a normal script at the bottom, so its event listener is registered after the document markup exists. The KPI section has initial placeholder content which is replaced by the first API response. Canvas/table-body nodes are renderer targets. Accessibility includes labels, `aria-live`, status role, and search labels.

The HTML form action points to `/get_footfall`, but dashboard.js prevents submit and uses `/api/predict`. Without JS, the form does not provide the same interactive prediction route behavior unless native submission is otherwise triggered; inspect this distinction when debugging progressive enhancement.

`app/templates/landing.html` (106 lines)

Marketing/overview page. Jinja adds server-generated `hero_profile` to `window.METROINSIGHTS_PREVIEW` using `tojson | safe`, then loads `landing.js` and `theme.js` deferred. `tojson` is the important serialization step: it produces JavaScript-compatible JSON and escapes sensitive characters for embedding. The page's static SVG/chart markup is a target for `drawPreviewChart`; scroll-train elements match the landing CSS/JS selectors.

`app/templates/result.html` (298 lines)

Legacy synchronous result report rendered by `/get_footfall`. Its fields (`station`, `station_info`, `day`, `date`, `time`, `calendar_context`, `crowding_color`, `crowding_badge`, `explanation`, `hourly_footfall`, `time_footfall`, schedule, `profile_24h`, `travel_info`, etc.) are supplied by route code. Jinja conditions separate interchange/single-line labels and open/closed service display. Inline JS filters destination rows and creates a Chart.js demand chart. Its client-side search hides rows and updates count; it searches station and line text. Most Jinja variable output is escaped by default; the profile uses `tojson | safe`, so safe serialization is expected. Inline style values are built from mapped values; dynamic CSS data should remain validated.

The result page duplicates some dashboard visualization behavior. The dashboard API workflow and this legacy result template are two entry paths; the latter is not the same DOM or response rendering path as dashboard.js.

10. Debugging and test ideas

1. In DevTools Network, confirm POST `/api/predict`, JSON request keys, status and response fields before debugging rendering. 2. In Console, inspect Chart.js failures separately from fetch failures. 3. Compare response `scheduling.headline` with displayed KPI values to isolate backend vs DOM mapping. 4. Verify every template ID used in JS exists exactly once on the relevant page. 5. Test out-of-order requests with delayed responses to ensure stale response protection works. 6. Test malicious-looking station/search text with DOM text rendering after replacing `innerHTML` sinks. 7. Browser tests should cover date presets at month/year boundaries, time values with minutes, empty search results, closed service, one-line vs interchange, dark mode after charts exist, and reduced-motion behavior.

No browser test framework or automated tests are tracked. These are proposed tests, not existing coverage.

Appendix source blocks

Complete HTML and JavaScript sources are appended with original line numbers. Excerpts are consecutive within each file, so you can follow the code directly in the checkout. CSS is covered in the next volume by selector groups and cascade order; it is kept separate to preserve readable source code.

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete dashboard Jinja template `app/templates/index.html:1-234`

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>MetroInsights - Delhi Metro Forecasting</title>
    <script>try{document.documentElement.dataset.theme=localStorage.getItem('metroflow-theme')||'light'}catch(e){}</script>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/metro-theme.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard-polish.css') }}">
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
    <script>
        window.STATIONS_META = {{ stations_meta | tojson | safe }};
    </script>
</head>
<body>
    <!-- Header -->
    <header class="dmrc-header">
        <div class="header-content">
            <div class="brand-wrapper">
                <span class="brand-mark" aria-hidden="true"><svg viewBox="0 0 40 40"><path d="M8 29V13a7 7 0 0 1 7-7h10a7 7 0 0 1 7 7v16H8Z"/><path d="M12 13h6v8h-6zm10 0h6v8h-6zM12 25h16M12 33l3-4m13 4-3-4"/><circle cx="14" cy="29" r="1"/><circle cx="26" cy="29" r="1"/></svg></span>
                <div class="brand-text">
                    <h1>MetroInsights <span>Forecasting workspace</span></h1>
                    <p>Delhi Metro demand scenarios \u00b7 Planning workspace</p>
                </div>
            </div>
            <div class="header-actions">
                <a class="header-home-link" href="/">Overview</a>
                <button class="theme-toggle" type="button" data-theme-toggle aria-label="Switch to dark theme" title="Switch to dark theme"><span class="theme-icon" aria-hidden="true"></span><span class="theme-toggle-label">Dark mode</span></button>
                <div class="status-pill">
                <span class="live-dot"></span>
                <span>Simulation mode</span>
                </div>
            </div>
        </div>
    </header>

    <div class="dashboard-container">

        <div class="dashboard-grid">
            <!-- Left Column: Controls & Model Info -->
            <div class="controls-column">
                <div class="card">
                    <div class="card-title">
                        <span>Forecast setup</span>
                        <span id="loadingIndicator" style="display: none; font-size: 0.75rem; color: #0284c7;">Calculating...</span>
                    </div>

                    <form action="/get_footfall" method="post" id="predictionForm">
                        <!-- Station Selection -->
                        <div class="form-group">
                            <label for="station">Select Metro Station</label>
                            <select name="station" id="station" class="form-select">
                                {% for st in stations %}
                                    <option value="{{ st }}" {% if st == default_station %}selected{% endif %}>
                                        {{ st }}{% if stations_meta[st].is_interchange %} \u21c4 [Interchange]{% endif %}
                                    </option>
                                {% endfor %}
                            </select>
                        </div>

                        <!-- Date Selection -->
                        <div class="form-group">
                            <label for="date">Date</label>
                            <input type="date" id="date" name="date" class="form-control" value="{{ default_date }}" min="{{ today_date }}">
                            <div class="quick-buttons">
                                <button type="button" class="btn-chip btn-preset-day active" data-preset="today">Today</button>
                                <button type="button" class="btn-chip btn-preset-day" data-preset="weekday">Next weekday</button>
                                <button type="button" class="btn-chip btn-preset-day" data-preset="saturday">Saturday</button>
                                <button type="button" class="btn-chip btn-preset-day" data-preset="sunday">Sunday</button>
                            </div>
                        </div>

                        <!-- Time Selection & Slider -->
                        <div class="form-group">
                            <label for="time">Operating Time: <span id="selectedTimeLabel" style="color: #0284c7; font-weight: 700;">{{ default_time }}</span></label>
                            <input type="time" id="time" name="time" class="form-control" value="{{ default_time }}">
                            <div class="slider-container">
                                <input type="range" id="timeSlider" min="5" max="23" value="9" step="1" style="width: 100%;">
                                <div class="slider-labels">
                                    <span>05:00</span>
                                    <span>09:00 (Rush)</span>
                                    <span>14:00</span>
                                    <span>18:00 (Rush)</span>
                                    <span>23:00</span>
                                </div>
                            </div>
                        </div>

                        <!-- Line Selection (for interchange stations) -->
                        <div class="form-group" id="lineSelectGroup" style="display: none;">
                            <label for="lineSelect">Corridor Focus</label>
                            <select name="line" id="lineSelect" class="form-select">
                                <option value="">All Lines (Station Overview)</option>
                            </select>
                        </div>

                        <div id="calendarContext" class="calendar-context" aria-live="polite">Loading date, workday, occasion, and demand period...</div>
                        <p id="forecastFormStatus" class="forecast-form-status" role="status" aria-live="polite"></p>
                    </form>
                </div>

            </div>

            <!-- Right Column: Live Results & Analytics -->
            <div class="results-column">
                <!-- Station Title & Summary -->
                <div class="card" style="margin-bottom: 20px;">
                    <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px;">
```

```
                        <div>
                            <h2 id="dispStationName" style="font-size: 1.5rem; color: #003366;">Rajiv Chowk</h2>
                            <p id="dispStationLine" style="color: #64748b; font-size: 0.9rem;">Interchange Hub (Yellow Line, Blue Line)</p>
                            <p id="dispForecastContext" class="forecast-context-line"></p>
                        </div>
                    </div>

                    <!-- Operational Recommendation Narrative -->
                    <div class="explanation-box">
                        <div class="explanation-title">
                            <span><svg class="section-icon" viewBox="0 0 20 20" aria-hidden="true"><path d="M5 3.5h10v13H5zM8 6.5h4M8 9.5h4M8 12.5h2"/><path d="M7 2v3m6-3v3"/></svg>Service recommendation</span>
                        </div>
                        <div id="dispExplanationText" class="explanation-body">
                            Existing baseline timetable comfortably handles projected passenger demand.
                        </div>
                    </div>
                </div>

                <!-- KPI Metric Grid -->
                <div class="kpi-grid">
                    <div class="kpi-card" style="border-left-color: #0284c7;">
                        <div class="kpi-title">Estimated passengers / hour</div>
                        <div id="dispFootfall" class="kpi-value">6,780</div>
                        <div id="dispInstantFootfall" class="kpi-subtext">Interpolated Minute: 6,780</div>
                    </div>

                    <div class="kpi-card" style="border-left-color: #10b981;">
                        <div class="kpi-title">Recommended Headway</div>
                        <div id="dispRecHeadway" class="kpi-value">2.3m</div>
                        <div id="dispExistingHeadway" class="kpi-subtext">Baseline: 2.3m (Peak)</div>
                    </div>

                    <div class="kpi-card" style="border-left-color: #8b5cf6;">
                        <div class="kpi-title">Recommended Service</div>
                        <div id="dispRecTrains" class="kpi-value">26.1</div>
                        <div id="dispExistingTrains" class="kpi-subtext">Baseline: 26.1 trains/hr</div>
                    </div>

                    <div class="kpi-card" style="border-left-color: #f59e0b;">
                        <div class="kpi-title">Capacity Utilization</div>
                        <div id="dispUtilization" class="kpi-value">20.7%</div>
                        <div id="dispUtilizationContext" class="kpi-subtext">Estimated under the recommended service plan</div>
                    </div>

                    <div class="kpi-card" style="border-left-color: #ef4444;">
                        <div class="kpi-title">Avg Passenger Wait</div>
                        <div id="dispWaitTime" class="kpi-value">1.2 min</div>
                        <div id="dispFleetDeployed" class="kpi-subtext">Active Fleet: 64/64 trains</div>
                    </div>
                </div>

                <!-- Operating-Hours Passenger Demand Curve -->
                <div class="card">
                    <div class="card-title">
                        <span>Passenger demand by operating hour</span>
                    </div>
                    <div class="chart-container">
                        <canvas id="demandChartCanvas"></canvas>
                    </div>
                </div>

                <!-- Metro Line Demand Distribution (Interchanges and Single Lines) -->
                <div class="card" id="interchangeSection">
                    <div class="card-title">
                        <span id="interchangeSectionTitle">Interchange Line Demand Distribution</span>
                    </div>
                    <p id="interchangeSectionDesc" style="font-size: 0.82rem; color: #64748b; margin-bottom: 12px;">
                        Demand distributed across intersecting lines using corridor capacity and network centrality weighting.
                    </p>
                    <div style="display: grid; grid-template-columns: 260px 1fr; gap: 20px; align-items: center;">
                        <div style="height: 200px;">
                            <canvas id="lineChartCanvas"></canvas>
                        </div>
                        <div>
                            <table class="data-table">
                                <thead>
                                    <tr>
                                        <th>Corridor</th>
                                        <th>Share</th>
                                        <th>Demand</th>
                                        <th>Base Headway</th>
                                        <th>Recommended</th>
                                        <th>Utilization</th>
                                    </tr>
                                </thead>
                                <tbody id="lineBreakdownTableBody">
                                    <!-- Populated by JavaScript -->
                                </tbody>
                            </table>
                        </div>
                    </div>
                </div>

                <!-- Connected Station Travel Times (Dijkstra) -->
                <div class="card">
                    <div class="card-title travel-card-title">
                        <div class="travel-title-group"><span>Travel times from</span><strong id="travelOriginName">Rajiv Chowk</strong></div>
                        <div class="travel-header-meta"><span class="travel-count"><span id="travelStationCount">226</span> destinations</span><span class="network-time-badge">Shortest path estimate</span></div>
                    </div>
                    <div class="travel-search-wrap">
                        <label class="travel-search-field"><svg viewBox="0 0 20 20" aria-hidden="true"><circle cx="8.5" cy="8.5" r="5.5"/><path d="m13 13 4 4"/></svg><input type="search" id="travelSearchInput" class="form-control" placeholder="Search a destination or corridor" aria-label="Search destination stations or corridors"></label>
                    </div>
                    <div class="travel-table-wrap">
                        <table class="data-table travel-table">
                            <thead>
                                <tr>
                                    <th>Destination station</th>
                                    <th>Station corridor</th>
                                    <th>Estimated travel time</th>
                                </tr>
                            </thead>
                            <tbody id="travelTimesTableBody">
                                <!-- Populated by JavaScript -->
```

```
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <script src="{{ url_for('static', filename='js/dashboard.js') }}"></script>
    <script src="{{ url_for('static', filename='js/theme.js') }}" defer></script>
</body>
</html>
```

Complete landing-page Jinja template `app/templates/landing.html:1-106`

```
<!doctype html>
<html lang="en" data-theme="light">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="theme-color" content="#f7f8fa">
    <meta name="description" content="Explore transparent Delhi Metro passenger demand estimates and service planning scenarios with MetroInsights.">
    <title>MetroInsights - Demand &amp;amp; Service Planning</title>
    <script>try{document.documentElement.dataset.theme=localStorage.getItem('metroflow-theme')||'light'}catch(e){}</script>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/metro-theme.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/landing.css') }}">
</head>
<body class="landing-page">
    <header class="site-header">
        <div class="site-header-inner">
            <a class="brand" href="/" aria-label="MetroInsights home">
                <span class="brand-mark" aria-hidden="true"><svg viewBox="0 0 40 40"><path d="M8 29V13a7 7 0 0 1 7-7h10a7 7 0 0 1 7 7v16H8Z"/><path d="M12 13h6v8h-6zm10 0h6v8h-6zM12 25h16M12 33l3-4m13 4-3-4"/><circle cx="14" cy="29" r="1"/><circle cx="26" cy="29" r="1"/></svg></span>
                <span><strong>MetroInsights</strong><small>Demand &amp;amp; service planning</small></span>
            </a>
            <nav class="site-nav" aria-label="Main navigation">
                <a href="#platform">Platform</a>
                <a href="#workflow">How it works</a>
                <a href="#transparency">Data &amp;amp; trust</a>
            </nav>
            <div class="site-actions">
                <button class="theme-toggle" type="button" data-theme-toggle aria-label="Switch to dark theme" title="Switch to dark theme"><span class="theme-icon" aria-hidden="true"></span><span class="theme-toggle-label">Dark mode</span></button>
                <a class="button button-primary header-cta" href="/dashboard">Explore forecasts <span aria-hidden="true">-></span></a>
                <button class="mobile-menu-toggle" type="button" aria-label="Open navigation" aria-expanded="false">\u2630</button>
            </div>
        </div>
        <div class="mobile-nav" hidden>
            <a href="#platform">Platform</a><a href="#workflow">How it works</a><a href="#transparency">Data &amp;amp; trust</a><a href="/dashboard">Open dashboard</a>
        </div>
    </header>

    <main>
        <section class="hero section-wrap" id="top">
            <div class="hero-copy reveal">
                <h1>Plan for the platform <span>before the crowd arrives.</span></h1>
                <p class="hero-lede">Explore station-level passenger demand estimates by date and hour, understand the day's travel context, and compare service recommendations in one calm, clear workspace.</p>
                <div class="hero-actions">
                    <a class="button button-primary button-large" href="/dashboard">Open forecasting workspace <span aria-hidden="true">-></span></a>
                    <a class="text-link" href="#platform">See how it works <span aria-hidden="true">\u2193</span></a>
                </div>
                <div class="hero-proof"><span class="proof-icon" aria-hidden="true">i</span><span>Simulation estimates are clearly labeled. They are not observed DMRC passenger counts.</span></div>
            </div>

            <div class="preview-shell reveal" aria-label="Example forecast preview">
                <div class="preview-topline"><span class="window-dots"><i></i><i></i><i></i></span><span>FORECAST PREVIEW</span><span class="preview-status"><b></b> Scenario estimate</span></div>
                <div class="preview-content">
                    <div class="preview-heading">
                        <div><span class="micro-label">STATION OVERVIEW</span><h2>{{ hero_station }}</h2><p>{{ default_date_label }} <span>\u00b7</span> 09:00</p></div>
                        <span class="interchange-chip">Interchange</span>
                    </div>
                    <div class="preview-stat-row">
                        <div class="preview-stat"><span>Hourly demand estimate</span><strong>{{ '{:,}'.format(hero_profile.predicted[4]) }}<small> riders</small></strong><em>Generated scenario</em></div>
                        <div class="preview-stat"><span>Typical station peak</span><strong>{{ hero_profile.peak_hour }}</strong><em>Based on selected date</em></div>
                    </div>
                    <div class="preview-chart-heading"><span>Operating day profile</span><span>05:00 - 23:00</span></div>
                    <div class="preview-chart-wrap"><svg class="preview-chart" viewBox="0 0 600 160" preserveAspectRatio="none" role="img" aria-label="Predicted station demand across operating hours"><defs><linearGradient id="areaFill" x1="0" y1="0" x2="0" y2="1"><stop offset="0%" stop-color="#2376a8" stop-opacity=".2"/><stop offset="100%" stop-color="#2376a8" stop-opacity="0"/></linearGradient></defs><path class="chart-area" d=""/><polyline class="chart-line" points=""/><line class="chart-cursor" x1="0" y1="15" x2="0" y2="142"/><circle class="chart-current" cx="0" cy="0" r="5"/></svg><div class="chart-xlabels"><span>05</span><span>09</span><span>14</span><span>18</span><span>23</span></div></div>
                    <div class="preview-route"><div class="route-label"><span class="route-dot"></span><span>NETWORK CONTEXT</span><span class="route-line-name">Blue \u00b7 Yellow</span></div><div class="route-track"><span class="route-track-fill"></span><span class="route-train" aria-hidden="true"><i></i><i></i><b></b></span><span class="route-stop stop-a"></span><span class="route-stop stop-b"></span><span class="route-stop stop-c"></span></div><div class="route-stations"><span>Rajiv Chowk</span><span>Central Secretariat</span><span>Botanical Garden</span></div></div>
                </div>
                <div class="preview-footnote">Illustrative model output \u00b7 synthetic training scenarios</div>
            </div>
            <div class="hero-background-route" aria-hidden="true"><span></span><span></span><span></span><span></span><span></span></div>
        </section>

        <div class="trust-strip"><div class="trust-inner"><span class="trust-lead">BUILT FOR CLEARER PLANNING</span><span class="trust-feature trust-blue"><i></i>Station-level estimates</span><span class="trust-feature trust-yellow"><i></i>Date-aware context</span><span class="trust-feature trust-green"><i></i>Service scenarios</span><span class="trust-feature trust-violet"><i></i>Transparent assumptions</span></div></div>

        <section class="section-wrap platform-section" id="platform">
            <div class="section-heading reveal"><span class="section-kicker">THE WORKSPACE</span><h2>From a date and station<br>to a planning picture.</h2><p>Move from a simple forecast question to the context behind the estimate and the service scenario it suggests.</p></div>
            <div class="feature-grid">
                <article class="feature-card reveal"><span class="feature-icon icon-calendar" aria-hidden="true">\u25a6</span><span class="feature-index">01 / CONTEXT</span><h3>Choose the day that matters</h3><p>Select today or a future date. The workspace identifies the weekday, workday status, and occasions included in its calendar.</p><span class="feature-line line-blue"></span></article>
                <article class="feature-card reveal"><span class="feature-icon icon-demand" aria-hidden="true">\u2301</span><span class="feature-index">02 / DEMAND</span><h3>Read the day by the hour</h3><p>Compare a station's estimated operating-hour profile, daily peak, and whether the selected hour is relatively busy or quiet.</p><span class="feature-line line-yellow"></span></article>
                <article class="feature-card reveal"><span class="feature-icon icon-service" aria-hidden="true">\u2194</span><span class="feature-index">03 / SERVICE</span><h3>Explore a service scenario</h3><p>Review headway, capacity, and wait-time recommendations with the assumptions that shape each result.</p><span class="feature-line line-red"></span></article>
            </div>
        </section>

        <section class="workflow-section" id="workflow">
            <div class="section-wrap workflow-inner">
```

```
                <div class="section-heading reveal"><span class="section-kicker">A SIMPLE FLOW</span><h2>Three steps to a clearer view.</h2><p>Keep the important choices up front. The supporting detail stays available when you need it.</p></div>
                <ol class="workflow-steps">
                    <li class="reveal"><span>1</span><div><h3>Set your scenario</h3><p>Choose a station, date, corridor, and operating hour.</p></div></li>
                    <li class="reveal"><span>2</span><div><h3>Understand the context</h3><p>See weekday, occasion, demand band, and the station's daily peak.</p></div></li>
                    <li class="reveal"><span>3</span><div><h3>Review the service picture</h3><p>Explore schedule and capacity recommendations alongside the forecast.</p></div></li>
                </ol>
                <div class="workflow-track" aria-hidden="true"><div class="workflow-track-line"></div><div class="workflow-train"><i></i><i></i><b></b></div><span class="workflow-station one"></span><span class="workflow-station two"></span><span class="workflow-station three"></span></div>
            </div>
        </section>

        <section class="transparency-section section-wrap" id="transparency">
            <div class="transparency-card reveal"><div class="transparency-icon" aria-hidden="true">\u25ce</div><div><span class="section-kicker">DATA &amp;amp; TRUST</span><h2>Clear assumptions.<br>Honest forecast context.</h2><p>The model learns from synthetic demand scenarios, with calendar, station, corridor, and service context. Operational values and line-level allocations are estimates. Holdout scores describe fit to simulated data; accuracy against observed DMRC passenger counts is unknown until station-by-hour counts are available.</p><div class="landing-data-notes"><span>Estimated operating assumptions</span><span>Synthetic demand scenarios</span><span>Not observed passenger counts</span></div></div><div class="transparency-badge"><span></span> Planning estimate</div></div>
        </section>

        <section class="closing-cta section-wrap"><div class="closing-copy reveal"><span class="section-kicker">READY TO EXPLORE?</span><h2>Start with a station.<br>See the whole picture.</h2><p>Open the forecasting workspace and shape a date-aware demand scenario.</p><a class="button button-primary button-large" href="/dashboard">Explore the workspace <span aria-hidden="true">-></span></a></div><div class="closing-map reveal" aria-hidden="true"><svg viewBox="0 0 360 240"><path class="map-line map-blue" d="M28 205 C90 205 83 145 145 145 S220 145 235 85 S290 35 330 35"/><path class="map-line map-yellow" d="M70 25 C70 90 145 85 145 145 S180 215 255 215"/><circle cx="145" cy="145" r="9"/><circle cx="235" cy="85" r="6"/><circle cx="70" cy="90" r="6"/><circle cx="255" cy="215" r="6"/><rect class="map-train" x="103" y="137" width="42" height="16" rx="7"/><path class="train-window" d="M111 141h9v7h-9zm13 0h9v7h-9z"/></svg></div></section>
    </main>

    <footer class="site-footer"><div class="footer-inner"><a class="brand footer-brand" href="/" aria-label="MetroInsights home"><span class="brand-mark" aria-hidden="true"><svg viewBox="0 0 40 40"><path d="M8 29V13a7 7 0 0 1 7-7h10a7 7 0 0 1 7 7v16H8Z"/><path d="M12 13h6v8h-6zm10 0h6v8h-6zM12 25h16M12 33l3-4m13 4-3-4"/><circle cx="14" cy="29" r="1"/><circle cx="26" cy="29" r="1"/></svg></span><span><strong>MetroInsights</strong><small>Demand &amp;amp; service planning</small></span></a><p>Planning estimates with transparent scenario assumptions</p><a href="/dashboard">Open workspace <span aria-hidden="true">-></span></a></div></footer>

    <div class="scroll-metro" aria-hidden="true"><div class="scroll-track"><span class="scroll-track-progress"></span><span class="scroll-station s1"></span><span class="scroll-station s2"></span><span class="scroll-station s3"></span><span class="scroll-station s4"></span><span class="scroll-train"><i></i><i></i><b></b></span></div></div>
    <script>window.METROINSIGHTS_PREVIEW = {{ hero_profile | tojson | safe }};</script>
    <script src="{{ url_for('static', filename='js/landing.js') }}" defer></script>
    <script src="{{ url_for('static', filename='js/theme.js') }}" defer></script>
</body>
</html>
```

Complete legacy result Jinja template and inline scripts `app/templates/result.html:1-298`

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Scenario recommendation: {{ station }} - MetroInsights</title>
    <script>try{document.documentElement.dataset.theme=localStorage.getItem('metroflow-theme')||'light'}catch(e){}</script>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/metro-theme.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard-polish.css') }}">
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
</head>
<body>
    <header class="dmrc-header">
        <div class="header-content">
            <div class="brand-wrapper">
                <span class="brand-mark" aria-hidden="true"><svg viewBox="0 0 40 40"><path d="M8 29V13a7 7 0 0 1 7-7h10a7 7 0 0 1 7 7v16H8Z"/><path d="M12 13h6v8h-6zm10 0h6v8h-6zM12 25h16M12 33l3-4m13 4-3-4"/><circle cx="14" cy="29" r="1"/><circle cx="26" cy="29" r="1"/></svg></span>
                <div class="brand-text">
                    <h1>MetroInsights <span>Scenario report</span></h1>
                    <p>Station: {{ station }} \u00b7 Simulation estimate</p>
                </div>
            </div>
            <div class="header-actions"><a class="header-home-link" href="/">Overview</a><a href="/dashboard" class="btn-chip" style="font-weight: 700; text-decoration: none; padding: 8px 16px;"><- Back to workspace</a><button class="theme-toggle" type="button" data-theme-toggle aria-label="Switch to dark theme" title="Switch to dark theme"><span class="theme-icon" aria-hidden="true"></span><span class="theme-toggle-label">Dark mode</span></button></div>
        </div>
    </header>

    <div class="dashboard-container">
        <!-- Station Summary Card -->
        <div class="card">
            <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px;">
                <div>
                    <h2 style="font-size: 1.8rem; color: #003366;">{{ station }}</h2>
                    <p style="color: #64748b; font-size: 0.95rem;">
                        {% if station_info.is_interchange %}
                            Interchange Hub: {{ station_info.all_lines | join(', ') }}
                        {% else %}
                            Corridor: {{ station_info.line }}
                        {% endif %}
                        * Evaluated for <strong>{{ day }}, {{ date }} at {{ time }}</strong>
                        * {{ calendar_context.day_status }}
                        * Occasion: <strong>{{ calendar_context.occasion or 'No major occasion in the loaded calendar' }}</strong>
                        * Forecast time band: <strong>{{ selected_hour_band }}</strong> (relative to this station/date profile)
                    </p>
                    {% if calendar_context.calendar_source %}<small>Occasion calendar: {{ calendar_context.calendar_source }}</small>{% endif %}
                </div>
                <div class="badge" style="background-color: {{ crowding_color }}; color: white; font-size: 0.9rem; padding: 8px 16px;">
                    {{ crowding_badge }}
                </div>
            </div>

            <!-- Operational Recommendation Narrative -->
            <div class="explanation-box">
                <div class="explanation-title">
                    <span>\U0001f4cb Scenario Schedule Recommendation:</span>
                </div>
                <div class="explanation-body">
                    {{ explanation }}
                </div>
            </div>
        </div>

        <!-- KPI Metrics Grid -->
        <div class="kpi-grid">
            <div class="kpi-card" style="border-left-color: #0284c7;">
                <div class="kpi-title">Hourly Demand</div>
                <div class="kpi-value">{{ hourly_footfall | int }}</div>
                <div class="kpi-subtext">Interpolated Minute: {{ time_footfall | int }} riders</div>
            </div>

            <div class="kpi-card" style="border-left-color: #10b981;">
                <div class="kpi-title">Recommended Headway</div>
                <div class="kpi-value">{{ (interval|string ~ 'm') if service_open else 'Closed' }}</div>
                <div class="kpi-subtext">{{ ('Baseline: ' ~ existing_interval|string ~ 'm') if service_open else 'No service at selected time' }}</div>
            </div>

            <div class="kpi-card" style="border-left-color: #8b5cf6;">
                <div class="kpi-title">Recommended Service</div>
                <div class="kpi-value">{{ num_trains_needed if service_open else 0 }}</div>
                <div class="kpi-subtext">{{ ('Baseline: ' ~ existing_trains|string ~ ' trains/hr') if service_open else 'No trains operating' }}</div>
            </div>

            <div class="kpi-card" style="border-left-color: #f59e0b;">
                <div class="kpi-title">Capacity Utilization</div>
                <div class="kpi-value">{{ (capacity_utilization|string ~ '%') if service_open else 'N/A' }}</div>
                <div class="kpi-subtext">{{ 'Comfort Clearance Budget' if service_open else 'Metro closed' }}</div>
            </div>

            <div class="kpi-card" style="border-left-color: #ef4444;">
                <div class="kpi-title">Avg Passenger Wait</div>
                <div class="kpi-value">{{ (avg_wait_time|string ~ ' min') if service_open else 'N/A' }}</div>
                <div class="kpi-subtext">{{ 'Random Arrival Standard' if service_open else 'No passenger service' }}</div>
            </div>
        </div>

        <!-- Line Breakdown for Interchanges & Single Corridors -->
        <div class="card">
            <div class="card-title">
                <span>{% if schedule_res.is_interchange %}Interchange Line Demand Distribution{% else %}Corridor Demand & Service Allocation{% endif %}</span>
            </div>
            <p style="font-size: 0.85rem; color: #64748b; margin-bottom: 12px;">
                {% if not service_open %}
                    Metro service is closed at the selected time; modeled demand and train service are zero.
                {% elif schedule_res.is_interchange %}
                    Demand distributed across intersecting lines based on corridor capacity and network centrality weighting.
                {% else %}
                    Full passenger demand allocated to this single-line corridor.
```

```
                {% endif %}
            </p>
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Corridor</th>
                        <th>Demand Share</th>
                        <th>Hourly Demand</th>
                        <th>Baseline Headway</th>
                        <th>Recommended Headway</th>
                        <th>Trains / Hour</th>
                        <th>Capacity Utilization</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    {% for l in schedule_res.line_recommendations %}
                    {% set color = '#dc2626' if 'Red' in l.line else ('#eab308' if 'Yellow' in l.line else ('#2563eb' if 'Blue' in l.line else ('#16a34a' if 'Green' in l.line else ('#7c3aed' if 'Violet' in l.line else ('#ec4899' if 'Pink' in l.line else ('#a855f7' if 'Magenta' in l.line else ('#ea580c' if 'Orange' in l.line else '#64748b')))))) %}
                    <tr>
                        <td>
                            <span style="display:inline-block; width:12px; height:12px; border-radius:3px; background-color:{{ color }}; vertical-align:middle; margin-right:8px;"></span>
                            <strong style="color: {{ '#b45309' if color == '#eab308' else color }};">{{ l.line }}</strong>
                        </td>
                        <td><strong>{{ (l.demand_share_pct|string ~ '%') if service_open else '-' }}</strong></td>
                        <td><strong>{{ l.line_demand | int }}</strong> riders/hr</td>
                        <td>{{ (l.existing_headway_min|string ~ ' min') if service_open else 'Closed' }}</td>
                        <td><strong style="color: #0284c7;">{{ (l.recommended_headway_min|string ~ ' min') if service_open else 'Closed' }}</strong></td>
                        <td>{{ l.recommended_trains_per_hour if service_open else 0 }} tph</td>
                        <td>
                            <span class="badge" style="background: {{ l.crowding_color }}; color: white;">{{ (l.capacity_utilization_pct|string ~ '%') if service_open else '-' }}</span>
                        </td>
                        <td>{{ 'Metro closed' if not service_open else l.status_action }}</td>
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>

        <!-- Operating-Hours Passenger Demand Profile Curve -->
        <div class="card">
            <div class="card-title">
                <span>Passenger demand by operating hour</span>
            </div>
            <div class="chart-container">
                <canvas id="demandResultChartCanvas"></canvas>
            </div>
        </div>

        <!-- Travel Times to Connected Stations (All Destinations) -->
        <div class="card">
            <div class="card-title">
                <span>Network Travel Times from {{ station }} (<span id="resultTravelCount" style="color: #0284c7;">{{ travel_info | length }}</span> Stations)</span>
                <span class="badge badge-verified">Dijkstra Shortest Path</span>
            </div>
            <div style="margin-bottom: 12px;">
                <input type="text" id="resultTravelSearch" class="form-control" placeholder="\U0001f50d Search any destination station (e.g. Kashmere Gate, Inderlok, Noida, Dwarka)...">
            </div>
            <div style="max-height: 480px; overflow-y: auto; border: 1px solid var(--border-color); border-radius: 6px;">
                <table class="data-table" id="resultTravelTable" style="margin-top: 0;">
                    <thead style="position: sticky; top: 0; background: #f8fafc; z-index: 2;">
                        <tr>
                            <th>Destination Station</th>
                            <th>Corridor Line</th>
                            <th>Estimated Transit Time</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for info in travel_info %}
                        {% set dest_line = info['Line'] if 'Line' in info else 'Metro Network' %}
                        {% set color = '#dc2626' if 'Red' in dest_line else ('#eab308' if 'Yellow' in dest_line else ('#2563eb' if 'Blue' in dest_line else ('#16a34a' if 'Green' in dest_line else ('#7c3aed' if 'Violet' in dest_line else ('#ec4899' if 'Pink' in dest_line else ('#a855f7' if 'Magenta' in dest_line else ('#ea580c' if 'Orange' in dest_line else '#64748b')))))) %}
                        <tr class="travel-row">
                            <td class="dest-name"><strong>{{ info['End Station'] }}</strong></td>
                            <td class="dest-line">
                                <span class="badge" style="background-color: {{ color }}; color: white; font-size: 0.75rem;">
                                    {{ dest_line }}
                                </span>
                            </td>
                            <td><strong style="color: #0284c7;">{{ info['Travel Time (minutes)'] }} mins</strong></td>
                        </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>

        <script>
            // Live search for destination stations
            const searchInput = document.getElementById('resultTravelSearch');
            if (searchInput) {
                searchInput.addEventListener('input', function() {
                    const q = this.value.toLowerCase().trim();
                    const rows = document.querySelectorAll('.travel-row');
                    let visibleCount = 0;
                    rows.forEach(r => {
                        const name = r.querySelector('.dest-name').textContent.toLowerCase();
                        const line = r.querySelector('.dest-line').textContent.toLowerCase();
                        if (!q || name.includes(q) || line.includes(q)) {
                            r.style.display = '';
                            visibleCount++;
                        } else {
                            r.style.display = 'none';
                        }
                    });
                    const countEl = document.getElementById('resultTravelCount');
                    if (countEl) countEl.textContent = visibleCount;
                });
            }
        </script>

        <div style="text-align: center; margin-top: 25px;">
```

```
            <a href="/" class="btn-primary" style="display: inline-block; width: auto; padding: 12px 30px; text-decoration: none;"><- Return to Dispatch Dashboard</a>
        </div>
    </div>

    <script>
        const profile = {{ profile_24h | tojson | safe }};
        const ctx = document.getElementById('demandResultChartCanvas');
        if (ctx && profile) {
            const datasets = [
                {
                    label: 'Simulated demand estimate (model trained on synthetic data)',
                    data: profile.predicted,
                    borderColor: '#0284c7',
                    backgroundColor: 'rgba(2, 132, 199, 0.08)',
                    borderWidth: 2.5,
                    fill: true,
                    tension: 0.35,
                    pointRadius: 4,
                    pointBackgroundColor: '#0284c7'
                },
                {
                    label: 'Synthetic training-period baseline mean',
                    data: profile.historical,
                    borderColor: '#94a3b8',
                    borderWidth: 1.8,
                    borderDash: [5, 5],
                    fill: false,
                    tension: 0.35,
                    pointRadius: 2
                }
            ];

            if (profile.line_curves && Object.keys(profile.line_curves).length > 1) {
                const METRO_LINE_COLORS = {
                    'Red Line': '#dc2626',
                    'Yellow Line': '#eab308',
                    'Blue Line': '#2563eb',
                    'Blue Line branch': '#0284c7',
                    'Green Line': '#16a34a',
                    'Green Line branch': '#22c55e',
                    'Violet Line': '#7c3aed',
                    'Pink Line': '#ec4899',
                    'Magenta Line': '#a855f7',
                    'Orange Line': '#ea580c',
                    'Grey Line': '#64748b'
                };
                for (const [lineName, curve] of Object.entries(profile.line_curves)) {
                    const color = METRO_LINE_COLORS[lineName] || '#0284c7';
                    datasets.push({
                        label: `${lineName} Demand Trend`,
                        data: curve,
                        borderColor: color,
                        borderWidth: 2,
                        borderDash: [4, 2],
                        fill: false,
                        tension: 0.35,
                        pointRadius: 2,
                        pointBackgroundColor: color
                    });
                }
            }

            new Chart(ctx, {
                type: 'line',
                data: {
                    labels: profile.labels,
                    datasets: datasets
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: {
                        y: { beginAtZero: true, title: { display: true, text: 'Passengers / Hour' } },
                        x: { title: { display: true, text: 'Operating Hour' } }
                    }
                }
            });
        }
    </script>
    <script src="{{ url_for('static', filename='js/theme.js') }}" defer></script>
</body>
</html>
```

Complete dashboard client JavaScript `app/static/js/dashboard.js:1-544`

```
/**
 * DMRC Intelligent Demand & Scheduling Interactive Client
 */

const METRO_LINE_COLORS = {
    'Red Line': '#e32017',
    'Yellow Line': '#e5b500',
    'Blue Line': '#0066b3',
    'Blue Line branch': '#0066b3',
    'Green Line': '#00843d',
    'Green Line branch': '#00843d',
    'Violet Line': '#7b2d5b',
    'Pink Line': '#df1683',
    'Magenta Line': '#8c1d82',
    'Orange Line': '#ed7d31',
    'Grey Line': '#6c7074',
    'Aqua Line': '#00a6a6'
};

function getLineColor(lineName) {
    if (!lineName) return '#0284c7';
    for (const [key, color] of Object.entries(METRO_LINE_COLORS)) {
        if (lineName.toLowerCase().includes(key.toLowerCase()) || key.toLowerCase().includes(lineName.toLowerCase())) {
            return color;
        }
    }
    return '#0284c7';
}

let demandChart = null;
let lineChart = null;
let currentData = null;
let currentTravelTimes = [];
let predictionRequestId = 0;

document.addEventListener('DOMContentLoaded', () => {
    initControls();
    fetchPrediction(); // Populate the analysis for the default scenario.
});

function initControls() {
    const stationSelect = document.getElementById('station');
    const timeSlider = document.getElementById('timeSlider');
    const timeInput = document.getElementById('time');
    const dateInput = document.getElementById('date');
    const lineSelect = document.getElementById('lineSelect');
    const travelSearchInput = document.getElementById('travelSearchInput');
    const predictionForm = document.getElementById('predictionForm');
    if (predictionForm) predictionForm.addEventListener('submit', (event) => event.preventDefault());

    // Station selection change
    if (stationSelect) {
        stationSelect.addEventListener('change', () => {
            updateLineSelectOptions();
            fetchPrediction();
        });
    }

    // Slider change
    if (timeSlider) {
        timeSlider.addEventListener('input', (e) => {
            const h = String(e.target.value).padStart(2, '0');
            if (timeInput) timeInput.value = `${h}:00`;
            document.getElementById('selectedTimeLabel').textContent = `${h}:00`;
            fetchPrediction();
        });
    }

    // Time input change
    if (timeInput) {
        timeInput.addEventListener('change', (e) => {
            const parts = e.target.value.split(':');
            const h = parseInt(parts[0], 10);
            if (timeSlider && !isNaN(h)) {
                timeSlider.value = h;
                document.getElementById('selectedTimeLabel').textContent = e.target.value;
            }
            fetchPrediction();
        });
    }

    // Date change
    if (dateInput) {
        dateInput.addEventListener('change', () => {
            document.querySelectorAll('.btn-preset-day').forEach(b => b.classList.remove('active'));
            fetchPrediction();
        });
    }

    // Line selection change
    if (lineSelect) {
        lineSelect.addEventListener('change', () => {
            fetchPrediction();
        });
    }

    // Travel station search input filter
    if (travelSearchInput) {
        travelSearchInput.addEventListener('input', () => {
            filterAndDisplayTravelTimes();
        });
    }

    // Quick Day Preset buttons
    document.querySelectorAll('.btn-preset-day').forEach(btn => {
        btn.addEventListener('click', (e) => {
            e.preventDefault();
            const preset = btn.getAttribute('data-preset');
            if (dateInput) {
                const target = new Date();
                target.setHours(12, 0, 0, 0);
                if (preset === 'weekday') {
                    do { target.setDate(target.getDate() + 1); }
                    while (target.getDay() === 0 || target.getDay() === 6);
                } else if (preset === 'saturday' || preset === 'sunday') {
                    const wantedDay = preset === 'saturday' ? 6 : 0;
```

```
                    do { target.setDate(target.getDate() + 1); }
                    while (target.getDay() !== wantedDay);
                }
                const targetDate = [target.getFullYear(), String(target.getMonth() + 1).padStart(2, '0'),
String(target.getDate()).padStart(2, '0')].join('-');
                dateInput.value = targetDate;
                document.querySelectorAll('.btn-preset-day').forEach(b => b.classList.remove('active'));
                btn.classList.add('active');
                fetchPrediction();
            }
        });
    });

    updateLineSelectOptions();
}

function updateLineSelectOptions() {
    const stationSelect = document.getElementById('station');
    const lineSelect = document.getElementById('lineSelect');
    if (!stationSelect || !lineSelect || !window.STATIONS_META) return;

    const selectedStation = stationSelect.value;
    const meta = window.STATIONS_META[selectedStation];

    lineSelect.innerHTML = '<option value="">All Lines (Station Overview)</option>';
    if (meta && meta.all_lines && meta.all_lines.length > 0) {
        meta.all_lines.forEach(line => {
            const opt = document.createElement('option');
            opt.value = line;
            opt.textContent = line;
            lineSelect.appendChild(opt);
        });
        document.getElementById('lineSelectGroup').style.display = 'block';
    } else {
        document.getElementById('lineSelectGroup').style.display = 'none';
    }
}

async function fetchPrediction() {
    const requestId = ++predictionRequestId;
    const station = document.getElementById('station').value;
    const date = document.getElementById('date').value;
    const time = document.getElementById('time').value;
    const line = document.getElementById('lineSelect').value;

    const loader = document.getElementById('loadingIndicator');
    const formStatus = document.getElementById('forecastFormStatus');
    if (loader) loader.style.display = 'inline-block';
    if (formStatus) {
        formStatus.textContent = 'Updating analysis...';
        formStatus.classList.remove('is-error');
    }

    try {
        const res = await fetch('/api/predict', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ station, date, time, line })
        });
        const data = await res.json();
        if (requestId !== predictionRequestId) return;
        if (data.status === 'success') {
            currentData = data;
            renderDashboard(data);
            document.body.classList.add('forecast-ready');
            if (formStatus) formStatus.textContent = '';
            requestAnimationFrame(() => {
                if (demandChart) demandChart.resize();
                if (lineChart) lineChart.resize();
            });
        } else if (formStatus) {
            formStatus.textContent = data.message || 'The forecast could not be generated. Check the selected inputs and try
again.';
            formStatus.classList.add('is-error');
        }
    } catch (err) {
        console.error('Error fetching prediction:', err);
        if (formStatus) {
            formStatus.textContent = 'We could not reach the forecasting service. Please try again.';
            formStatus.classList.add('is-error');
        }
    } finally {
        if (requestId === predictionRequestId) {
            if (loader) loader.style.display = 'none';
        }
    }
}

function renderDashboard(data) {
    if (!data || !data.scheduling) return;
    const { station, datetime, footfall, scheduling, profile_24h, station_info } = data;
    const hl = scheduling.headline;
    const serviceClosed = datetime.service_open === false;
    const selectedLine = document.getElementById('lineSelect')?.value || '';
    const lineDemand = Number.isFinite(Number(hl.line_demand)) ? Number(hl.line_demand) : footfall.hourly_demand;
    const originEl = document.getElementById('travelOriginName');
    if (originEl) originEl.textContent = station;
    const contextLine = document.getElementById('dispForecastContext');
    if (contextLine) {
        const calendar = data.calendar_context || {};
        const dateLabel = new Date(`${datetime.date}T12:00:00`).toLocaleDateString('en-IN', { weekday: 'long', day: 'numeric',
month: 'long', year: 'numeric' });
        const occasion = calendar.occasion || 'No major occasion in the loaded calendar';
        contextLine.textContent = `${dateLabel} \u00b7 ${calendar.day_status || 'Day type unavailable'} \u00b7 ${datetime.time}
\u00b7 ${occasion}`;
    }

    const calendarEl = document.getElementById('calendarContext');
    if (calendarEl && data.calendar_context) {
        const calendar = data.calendar_context;
        const occasion = calendar.occasion || 'No major occasion in the loaded calendar';
        const source = calendar.calendar_source ? ` Calendar: ${calendar.calendar_source}.` : '';
        calendarEl.textContent = serviceClosed
            ? `${calendar.weekday}, ${datetime.date} \u00b7 ${calendar.day_status} \u00b7 ${occasion}. ${datetime.time}: Metro
closed; no trains or passenger demand are modeled. Daily operating-hours peak ${calendar.daily_peak_hour}.${source}`
            : `${calendar.weekday}, ${datetime.date} \u00b7 ${calendar.day_status} \u00b7 ${occasion}. ${datetime.time}:
${calendar.hour_band}, relative to this station's forecast curve; daily peak ${calendar.daily_peak_hour}.${source}`;
    }


```

```
    try {
        // Station badges & details
        const stNameEl = document.getElementById('dispStationName');
        if (stNameEl) stNameEl.textContent = station;
        const stLineEl = document.getElementById('dispStationLine');
        if (stLineEl) {
            stLineEl.textContent = station_info.is_interchange
                ? `Interchange Hub (${station_info.all_lines.join(', ')})`
                : station_info.line;
        }

        // KPI 1: Hourly Demand
        const ffEl = document.getElementById('dispFootfall');
        if (ffEl) ffEl.textContent = lineDemand.toLocaleString();
        const instFfEl = document.getElementById('dispInstantFootfall');
        const currentShare = footfall.hourly_demand ? lineDemand / footfall.hourly_demand : 1;
        if (instFfEl) instFfEl.textContent = serviceClosed ? 'Metro closed; no passenger service' : `${selectedLine ? `${selectedLine} \u00b7 ` : (hl.is_station_summary ? 'All lines \u00b7 ' : '')}Interpolated estimate: ${Math.round(footfall.instant_interpolated * currentShare).toLocaleString()}`;

        // KPI 2: Recommended Headway
        const recHwEl = document.getElementById('dispRecHeadway');
        if (recHwEl) recHwEl.textContent = serviceClosed ? 'Closed' : (hl.is_station_summary ? 'Varies' : `${hl.recommended_headway_min}m`);
        const existHwEl = document.getElementById('dispExistingHeadway');
        if (existHwEl) existHwEl.textContent = serviceClosed ? 'No service at selected time' : (hl.is_station_summary ? `See ${hl.line_count} line recommendations below` : `Baseline: ${hl.existing_headway_min}m (${hl.is_peak ? 'Peak' : 'Off-Peak'})`);

        // KPI 3: Trains Per Hour
        const recTrEl = document.getElementById('dispRecTrains');
        if (recTrEl) recTrEl.textContent = serviceClosed ? '0' : `${hl.recommended_trains_per_hour}`;
        const existTrEl = document.getElementById('dispExistingTrains');
        if (existTrEl) existTrEl.textContent = serviceClosed ? 'No trains operating' : (hl.is_station_summary ? `Combined across ${hl.line_count} lines \u00b7 trains/hour` : `Baseline: ${hl.existing_trains_per_hour} trains/hr`);

        // KPI 4: Capacity Utilization & Crowding Risk
        const utilEl = document.getElementById('dispUtilization');
        if (utilEl) utilEl.textContent = serviceClosed ? 'N/A' : `${hl.capacity_utilization_pct}%`;
        const utilContextEl = document.getElementById('dispUtilizationContext');
        if (utilContextEl) {
            const scope = hl.is_station_summary ? 'All lines' : (selectedLine || hl.line);
            const capacity = Number(hl.recommended_clearance_capacity || 0).toLocaleString();
            utilContextEl.textContent = serviceClosed ? 'Metro closed' : `${scope} \u00b7 estimated boarding capacity ${capacity}/hr`;
        }
        // KPI 5: Average Wait Time
        const waitEl = document.getElementById('dispWaitTime');
        if (waitEl) waitEl.textContent = serviceClosed ? 'N/A' : `${hl.avg_wait_time_rec_min} min`;
        const fleetEl = document.getElementById('dispFleetDeployed');
        if (fleetEl) fleetEl.textContent = serviceClosed ? 'No trains operating' : `Active Fleet: ${hl.trains_deployed_rec}/${hl.fleet_size_available} trains`;

        // Operational Explanation Box
        const expEl = document.getElementById('dispExplanationText');
        if (expEl) expEl.textContent = hl.explanation;
    } catch (e) {
        console.error('Error rendering KPI elements:', e);
    }

    try {
        // Render 24h Demand Curve Chart
        let demandProfile = profile_24h;
        if (selectedLine && profile_24h.line_curves && profile_24h.line_curves[selectedLine]) {
            const share = footfall.hourly_demand ? lineDemand / footfall.hourly_demand : 1;
            demandProfile = {
                ...profile_24h,
                predicted: profile_24h.line_curves[selectedLine],
                historical: (profile_24h.historical || []).map(value => Math.round(value * share))
            };
        }
        const chartTitle = document.querySelector('#demandChartCanvas')?.closest('.card')?.querySelector('.card-title span');
        if (chartTitle) chartTitle.textContent = selectedLine ? `${selectedLine} passenger demand by hour` : 'Passenger demand by operating hour';
        renderDemandChart(demandProfile, datetime.hour);
    } catch (e) {
        console.error('Error rendering demand chart:', e);
    }

    try {
        // Render Line Breakdown (Always shown, whether interchange or single line!)
        renderInterchangeBreakdown(scheduling);
    } catch (e) {
        console.error('Error rendering line breakdown:', e);
    }

    try {
        // Render Travel Times table (All 226 stations)
        renderTravelTimes(data.travel_times);
    } catch (e) {
        console.error('Error rendering travel times:', e);
    }
}

function renderDemandChart(profile, currentHour) {
    const ctx = document.getElementById('demandChartCanvas');
    if (!ctx || !profile) return;

    const labels = profile.labels || [];
    const predicted = profile.predicted || [];
    const historical = profile.historical || [];

    // Mark current selected hour point
    const pointRadii = labels.map((_, i) => profile.hours && profile.hours[i] === currentHour ? 6 : 3);
    const pointBackgrounds = labels.map((_, i) => profile.hours && profile.hours[i] === currentHour ? '#ef4444' : '#0284c7');

    const datasets = [
        {
            label: 'Simulated demand estimate (model trained on synthetic data)',
            data: predicted,
            borderColor: '#0284c7',
            backgroundColor: 'rgba(2, 132, 199, 0.08)',
            borderWidth: 2.5,
            fill: true,
            tension: 0.35,
            pointRadius: pointRadii,
            pointBackgroundColor: pointBackgrounds
        },
        {
            label: 'Synthetic training-period baseline mean',
```

```
            data: historical,
            borderColor: '#94a3b8',
            borderWidth: 1.8,
            borderDash: [5, 5],
            fill: false,
            tension: 0.35,
            pointRadius: 2
        }
    ];

    // If interchange with multiple lines, add each corridor's 24-hour demand trend
    if (profile.line_curves && Object.keys(profile.line_curves).length > 1) {
        for (const [lineName, curve] of Object.entries(profile.line_curves)) {
            const lineColor = getLineColor(lineName);
            datasets.push({
                label: `${lineName} Demand Trend`,
                data: curve,
                borderColor: lineColor,
                backgroundColor: 'transparent',
                borderWidth: 2,
                borderDash: [4, 2],
                fill: false,
                tension: 0.35,
                pointRadius: 2,
                pointBackgroundColor: lineColor
            });
        }
    }

    if (demandChart) {
        demandChart.destroy();
    }

    demandChart = new Chart(ctx, {
        type: 'line',
        data: {
            labels: labels,
            datasets: datasets
        },
        options: {
            responsive: true,
            maintainAspectRatio: false,
            plugins: {
                legend: {
                    position: 'top',
                    labels: { font: { size: 12 } }
                },
                tooltip: {
                    callbacks: {
                        label: (ctx) => `${ctx.dataset.label}: ${ctx.raw.toLocaleString()} passengers`
                    }
                }
            },
            scales: {
                y: {
                    beginAtZero: true,
                    title: { display: true, text: 'Passengers / Hour', font: { size: 12 } }
                },
                x: {
                    title: { display: true, text: 'Operating Hour (05:00 - 23:00)', font: { size: 12 } }
                }
            }
        }
    });
}

function renderInterchangeBreakdown(scheduling) {
    const container = document.getElementById('interchangeSection');
    if (!container) return;

    // ALWAYS visible for all stations
    container.style.display = 'block';

    const titleEl = document.getElementById('interchangeSectionTitle');
    const descEl = document.getElementById('interchangeSectionDesc');

    if (scheduling.is_interchange) {
        if (titleEl) titleEl.textContent = 'Interchange Line Demand Distribution';
        if (descEl) descEl.textContent = 'Demand distributed across intersecting lines using corridor capacity and network centrality weighting.';
    } else {
        if (titleEl) titleEl.textContent = 'Corridor Demand & Service Allocation';
        if (descEl) descEl.textContent = 'Full passenger demand allocated to this single-line corridor.';
    }

    const lines = scheduling.all_lines;
    const tableBody = document.getElementById('lineBreakdownTableBody');
    tableBody.innerHTML = '';

    const labels = [];
    const demands = [];
    const sliceColors = [];

    lines.forEach((l) => {
        labels.push(l.line);
        demands.push(l.line_demand);
        const lineColor = getLineColor(l.line);
        sliceColors.push(lineColor);

        const tr = document.createElement('tr');
        const serviceClosed = l.status_action === 'SERVICE_CLOSED';
        tr.innerHTML = `
            <td>
                <span class="line-chip" style="--metro-line-color: ${lineColor}"><i class="line-marker" aria-hidden="true"></i><span>${l.line}</span></span>
            </td>
            <td><strong>${serviceClosed ? '-' : `${l.demand_share_pct}%`}</strong></td>
            <td><strong>${l.line_demand.toLocaleString()}</strong> riders/hr</td>
            <td>${serviceClosed ? 'Closed' : `${l.existing_headway_min}m`}</td>
            <td><strong style="color: #0284c7;">${serviceClosed ? 'Closed' : `${l.recommended_headway_min}m (${l.recommended_trains_per_hour} tph)`}</strong></td>
            <td><span class="badge" style="background: ${l.crowding_color}; color: white;">${serviceClosed ? '-' : `${l.capacity_utilization_pct}%`}</span></td>
        `;
        tableBody.appendChild(tr);
    });

    const ctx = document.getElementById('lineChartCanvas');
    if (ctx) {
```

```
        if (lineChart) lineChart.destroy();
        lineChart = new Chart(ctx, {
            type: 'doughnut',
            data: {
                labels: labels,
                datasets: [{
                    data: demands,
                    backgroundColor: sliceColors,
                    borderWidth: 2,
                    borderColor: getComputedStyle(document.documentElement).getPropertyValue('--surface').trim() || '#ffffff'
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: {
                        position: 'bottom',
                        labels: {
                            font: { size: 12, weight: 'bold' },
                            usePointStyle: true,
                            pointStyle: 'rectRounded'
                        }
                    },
                    tooltip: {
                        callbacks: {
                            label: (ctx) => ` ${ctx.label}: ${ctx.raw.toLocaleString()} riders/hr`
                        }
                    }
                }
            }
        });
    }
}

function renderTravelTimes(times) {
    if (times) {
        currentTravelTimes = times;
    }
    filterAndDisplayTravelTimes();
}

function filterAndDisplayTravelTimes() {
    const tableBody = document.getElementById('travelTimesTableBody');
    const searchInput = document.getElementById('travelSearchInput');
    const countBadge = document.getElementById('travelStationCount');
    if (!tableBody) return;

    const query = searchInput ? searchInput.value.toLowerCase().trim() : '';

    const filtered = currentTravelTimes.filter(item => {
        if (!query) return true;
        const dest = (item.destination || item['End Station'] || '').toLowerCase();
        const line = (item.line || item['Line'] || '').toLowerCase();
        return dest.includes(query) || line.includes(query);
    });

    if (countBadge) {
        countBadge.textContent = `${filtered.length} of ${currentTravelTimes.length}`;
    }

    tableBody.innerHTML = '';

    if (filtered.length === 0) {
        const tr = document.createElement('tr');
        tr.innerHTML = `<td colspan="3" style="text-align: center; color: #64748b; padding: 20px;">No destination stations matching "${query}"</td>`;
        tableBody.appendChild(tr);
        return;
    }

    filtered.forEach(item => {
        const dest = item.destination || item['End Station'];
        const mins = item.travel_time_min ?? item['Travel Time (minutes)'];
        const line = item.line || item['Line'] || 'Metro Network';
        const lineColor = getLineColor(line);

        const tr = document.createElement('tr');
        tr.innerHTML = `
            <td><strong>${dest}</strong></td>
            <td>
                <span class="line-chip" style="--metro-line-color: ${lineColor}">
                    <i class="line-marker" aria-hidden="true"></i><span>${line}</span>
                </span>
            </td>
            <td><span class="travel-time-value">${mins}<small> min</small></span></td>
        `;
        tableBody.appendChild(tr);
    });
}
```

Complete landing animation and preview JavaScript `app/static/js/landing.js:1-89`

```
(function () {
  const profile = window.METROINSIGHTS_PREVIEW;

  function drawPreviewChart() {
    const svg = document.querySelector('.preview-chart');
    if (!svg || !profile || !Array.isArray(profile.predicted) || !profile.predicted.length) return;
    const values = profile.predicted;
    const max = Math.max(...values) || 1;
    const min = Math.min(...values);
    const spread = max - min || 1;
    const points = values.map((value, index) => {
      const x = 12 + (index / Math.max(values.length - 1, 1)) * 576;
      const y = 132 - ((value - min) / spread) * 100;
      return [x, y];
    });
    const line = points.map((point) => point.map((v) => v.toFixed(1)).join(',')).join(' ');
    const area = `M ${points[0][0]} 145 L ${points.map((p) => `${p[0].toFixed(1)} ${p[1].toFixed(1)}`).join(' L ')} L ${points[points.length - 1][0]} 145 Z`;
    svg.querySelector('.chart-line').setAttribute('points', line);
    svg.querySelector('.chart-area').setAttribute('d', area);
    const selectedIndex = 4; // The sample card is set to 09:00.
    const current = points[selectedIndex] || points[0];
    svg.querySelector('.chart-current').setAttribute('cx', current[0]);
    svg.querySelector('.chart-current').setAttribute('cy', current[1]);
    svg.querySelector('.chart-cursor').setAttribute('x1', current[0]);
    svg.querySelector('.chart-cursor').setAttribute('x2', current[0]);
  }

  function initScrollMetro() {
    const reducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
    const rail = document.querySelector('.scroll-track');
    const train = rail && rail.querySelector('.scroll-train');
    const progressLine = rail && rail.querySelector('.scroll-track-progress');
    const update = () => {
      if (!rail) return;
      const scrollable = document.documentElement.scrollHeight - window.innerHeight;
      const progress = scrollable > 0 ? Math.min(1, Math.max(0, window.scrollY / scrollable)) : 0;
      const travel = reducedMotion ? 0 : progress;
      if (train) train.style.top = `${travel * Math.max(0, rail.clientHeight - train.offsetHeight)}px`;
      if (progressLine) progressLine.style.height = `${travel * rail.clientHeight}px`;
    };
    update();
    window.addEventListener('scroll', update, { passive: true });
    window.addEventListener('resize', update);
  }

  function initReveals() {
    const items = document.querySelectorAll('.reveal');
    if (!('IntersectionObserver' in window) || window.matchMedia('(prefers-reduced-motion: reduce)').matches) {
      items.forEach((item) => item.classList.add('is-visible'));
      return;
    }
    document.documentElement.classList.add('motion-ready');
    const observer = new IntersectionObserver((entries, instance) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          entry.target.classList.add('is-visible');
          instance.unobserve(entry.target);
        }
      });
    }, { threshold: 0.12, rootMargin: '0px 0px -24px 0px' });
    items.forEach((item) => observer.observe(item));
  }

  function initMobileNavigation() {
    const toggle = document.querySelector('.mobile-menu-toggle');
    const menu = document.querySelector('.mobile-nav');
    if (!toggle || !menu) return;
    toggle.addEventListener('click', () => {
      const open = toggle.getAttribute('aria-expanded') !== 'true';
      toggle.setAttribute('aria-expanded', String(open));
      toggle.setAttribute('aria-label', open ? 'Close navigation' : 'Open navigation');
      toggle.textContent = open ? '\u00d7' : '\u2630';
      menu.hidden = !open;
    });
    menu.querySelectorAll('a').forEach((link) => link.addEventListener('click', () => {
      menu.hidden = true;
      toggle.setAttribute('aria-expanded', 'false');
      toggle.setAttribute('aria-label', 'Open navigation');
      toggle.textContent = '\u2630';
    }));
  }

  document.addEventListener('DOMContentLoaded', () => {
    drawPreviewChart();
    initScrollMetro();
    initReveals();
    initMobileNavigation();
  });
}());
```

Complete theme toggle JavaScript `app/static/js/theme.js:1-48`

```
(function () {
  const root = document.documentElement;
  const buttons = () => document.querySelectorAll('[data-theme-toggle]');
  const icons = {
    sun: '<svg viewBox="0 0 24 24" focusable="false"><circle cx="12" cy="12" r="4"/><path d="M12 2v2m0 16v2M4.93 4.93l1.42 1.42m11.3 11.3 1.42 1.42M2 12h2m16 0h2M4.93 19.07l1.42-1.42m11.3-11.3 1.42-1.42"/></svg>',
    moon: '<svg viewBox="0 0 24 24" focusable="false"><path d="M20.2 15.4A8.7 8.7 0 0 1 8.6 3.8 8.8 8.8 0 1 0 20.2 15.4Z"/></svg>'
  };

  function applyTheme(theme) {
    const activeTheme = theme === 'dark' ? 'dark' : 'light';
    root.dataset.theme = activeTheme;
    try { localStorage.setItem('metroflow-theme', activeTheme); } catch (_) { /* Theme still works for this page. */ }
    buttons().forEach((button) => {
      const next = activeTheme === 'dark' ? 'light' : 'dark';
      button.setAttribute('aria-label', `Switch to ${next} theme`);
      button.setAttribute('title', `Switch to ${next} theme`);
      const label = button.querySelector('.theme-toggle-label');
      if (label) label.textContent = `${next === 'dark' ? 'Dark' : 'Light'} mode`;
      const icon = button.querySelector('.theme-icon');
      if (icon) icon.innerHTML = activeTheme === 'dark' ? icons.sun : icons.moon;
    });
    if (window.Chart) {
      const chartText = activeTheme === 'dark' ? '#a4adb7' : '#657184';
      const chartGrid = activeTheme === 'dark' ? 'rgba(255, 255, 255, 0.07)' : 'rgba(32, 38, 49, 0.08)';
      window.Chart.defaults.color = chartText;
      window.Chart.defaults.borderColor = chartGrid;
      Object.values(window.Chart.instances || {}).forEach((chart) => {
        chart.options.color = chartText;
        Object.values(chart.options.scales || {}).forEach((scale) => {
          scale.ticks = { ...scale.ticks, color: chartText };
          scale.grid = { ...scale.grid, color: chartGrid };
          if (scale.title) scale.title = { ...scale.title, color: chartText };
        });
        if (chart.options.plugins?.legend?.labels) {
          chart.options.plugins.legend.labels.color = chartText;
        }
        chart.update('none');
      });
    }
  }

  document.addEventListener('DOMContentLoaded', () => {
    applyTheme(root.dataset.theme || 'light');
    buttons().forEach((button) => button.addEventListener('click', () => {
      applyTheme(root.dataset.theme === 'dark' ? 'light' : 'dark');
    }));
  });
}());
```